



IONITY GLOBAL

(Pty) Ltd

Gate^Flame Network Security Node

Pinned State Record — Resume Here

Deployment Readiness & Defect Disclosure

Raspberry Pi node agent | Android companion | LAN pairing

DOC-2026-08-013-STATE | Version 6.0 | 15 August 2026

INTERNAL

www.ionity.today | ai@ionity.today

Index – Table of Contents

This record supersedes v5.1 in full. Page numbers are indicative; actual pagination depends on rendering and may shift between Word and PDF.

Section	Topic	Page
1	Position statement	3
2	What changed in v6.0 — deployment was attempted and failed	3
3	The four defects	4
4	Commit ledger	5
5	Controlled artefacts on disk	5
6	Deployment procedure	6
7	Outstanding actions	7
8	Known gaps — not blocking	8
9	Verification record	8

1. Position statement

Repository: dennisGlonity/Gate-Flame. The remote is SSH. Remote references were verified against the live remote during this session:

- main = a0a2da3
- chore/repo-hygiene = 919856a

Gate^Flame is a network security appliance: a Raspberry Pi node running a Python agent, and an Android companion application that pairs with it over the customer's own LAN. As of this revision both halves have been built, packaged and verified to the limit of what can be proven without the hardware in hand.

2. What changed in v6.0 — deployment was attempted, and it failed

Version 5.1 listed “validate on a real Pi” and “load the APK on a phone” as the last two open items. That framing implied the code was ready and only hardware was missing. It was not.

Building and running the deployment path end to end surfaced four defects, each of which independently made first deployment impossible. All four are now fixed and verified.

Lesson worth carrying. Defects 1, 2 and 4 were all in code that type-checked, built clean and had passing tests. The suite stubbed fetch and never asserted a port; nothing exercised the Android manifest at all. “465 tests green” was a true statement that told us nothing about whether the product could reach its own hardware.

3. The four defects

#	Defect	Effect
1	Six of eight discoveryCandidates omitted :8080. The agent binds 8080; they resolved to :80.	The phone could find a node only over its own loopback. On a real LAN, always “No Gate^Flame node found”, with no manual-address path to recover.
2	AppPairingScreen never called runDiscovery.	It rendered “Looking for a Gate^Flame node...” while doing nothing. Only “Search again” ever probed the network.

#	Defect	Effect
3	android/gradle/wrapper/gradle-wrapper.jar corrupt since the initial public commit 67fed8 — 15,436 U+FFFD sequences, decodes as UTF-8. Same text-mode corruption class as the release tarball.	gradlew died with “Invalid or corrupt jarfile” on any fresh clone. No APK was buildable by anyone, including CI.
4	network_security_config.xml tried to scope cleartext to RFC1918 by listing 192.168.1.0 etc. with includeSubdomains. Android's <domain> is a hostname match with no CIDR support — 192.168.1.0 does not match 192.168.1.105.	Every request to a node by IP was killed on the handset before a socket opened. Only gateflame.local ever worked. Confirmed by aapt2 dump xmltree on the built APK.

3.1 A note on the fix for defect 4

“Permit cleartext” reads like a loosening, so it is worth stating plainly. No hostname list can express RFC1918 in that file. The manifest therefore permits cleartext, and the restriction moves to `assertPrivateHost()` in `src/services/apiClient.ts`, which every request passes through before a socket opens.

It admits RFC1918, RFC3927 link-local, loopback, .local mDNS names and any https URL, and refuses everything else — so a bearer token cannot leave the phone towards a public host in the clear. That is a stronger guarantee than the old file claimed, and unlike the old file it is tested: 17 cases across the edges most easily got wrong (172.15.255.255 refused, 172.16.0.5 admitted, 172.32.0.1 refused).

The node enforces the same boundary independently in `security.py`, which gates every route on the source address. Neither side relies on the other.

4. Commit ledger

Seven commits, 919856a → cfccc48.

Commit	Content
d5a1bcd	applicationId frozen: today.ionity.gateflame
6a8e1ba	nftables bouncer, keystore generator, Pi validator
1cf9b10	DPI, WAN budget, zero-trust posture, feed-receiver
13680e7	Code-splitting, 72 frontend tests, polling-loop fix, POPIA review, SECURITY.md
77f4819	DENNIS-DO-THIS.md action sheet
8c26a46	Defects 1–3, manual address entry, node-agent/deploy-on-pi.sh
cfccc48	Defect 4, <code>assertPrivateHost()</code> runtime guard, 17 guard tests

919856a..cfccc48 is a clean fast-forward, verified against the live remote. main still requires a pull-request merge rather than a push: a0a2da3 is a merge commit outside the branch's history.

5. Controlled artefacts on disk

Location: C:\Users\DGMeic\GateFlame-Backup-2026-08-13_fix-2026-08-13\

Artefact	Status
gateflame-2026-08-15-DEPLOY.bundle	Tip cfccc48 — USE THIS
GateFlame-Mobile-1.0.1-debug.apk	Installable, today.ionity.gateflame.debug
gateflame-pi-2026-08-15.tar.gz	Agent, one-shot installer, hardware validator

Artefact	Status
DEPLOY-RUNB00K.md	Current deployment instructions
SHA256SUMS.txt	All three digests verified on-device after transfer
gateflame-2026-08-15-FINAL.bundle	SUPERSEDED (tip 77f4819)
DENNIS-D0-THIS.md	Still accurate for items 4–9
ROTATE-ME.txt	Old Gemini key — revoke, then delete

6. Deployment procedure

6.1 Raspberry Pi node

Run as root on the Pi:

```
sudo bash deploy-on-pi.sh
```

Four stages, stopping at the first real failure: prerequisites; node-agent/install.sh; a systemd drop-in granting CAP_NET_ADMIN and CAP_NET_RAW plus an avahi alias publishing gateflame.local; then validate-on-pi.sh. It prints the node's LAN address and a full uninstall path. Every step is reversible.

The validator's PASS/FAIL table is the important output. It is the first honest answer this project has had to the question of whether its telemetry numbers are real hardware readings. Everything to date came from a container with no thermal zone, no vcgencmd and no real ARP table — proving only that the fallback paths work.

6.2 Android companion

Install the debug APK. The application now discovers the node by itself, with a manual address box as fallback.

6.3 Pairing

Pairing codes are issued only to loopback callers — physical presence at the appliance is the authorisation. There is no kiosk on the node, so issue a code over SSH:

```
curl -s -X POST http://127.0.0.1:8080/api/v1/pair/request
```

Codes live five minutes; five wrong guesses destroys the code, not the node. Verified end to end this session against a live agent: status → request → claim → real telemetry, with a real token issued.

7. Outstanding actions

#	Item	Status
1	Push 919856a → cfccc48, open and merge PR #2, tag v1.0.1	Ready — fast-forward verified
2	Run deploy-on-pi.sh on the Pi and read the PASS/FAIL table	THE REAL UNKNOWN — nothing has ever run on Pi hardware
3	Install the APK and pair over SSH	Ready
4	Revoke two GitHub personal access tokens	URGENT — disclosed 2026-08-14
5	Revoke GEMINI_API_KEY, delete ROTATE-ME.txt	URGENT — needs a Google login
6	Generate the release keystore (android/generate-keystore.ps1, Windows only)	URGENT — no recovery path if lost

#	Item	Status
7	POPIA review by a practitioner — docs/POPIA-REVIEW.md §5	Hosting location of feeds.ionity.today (s72) first

8. Known gaps — not blocking

- node-agent serves no static route, so the kiosk HTML is built but nothing serves it; /device-kiosk does not exist.
- build-standalone.js remains orphaned — no npm script calls it.
- No breach response plan (POPIA s22).
- No privacy notice published (POPIA s18).
- feed-receiver/ is written and tested but deployed nowhere.

9. Verification record

Suite	Tests
node-agent (pytest)	310
feed-receiver	83
Frontend (vitest)	90 — was 72
Total	483

tsc --noEmit is clean. Both HTML builds succeed. gradlew runs from a clean clone. assembleDebug produces a verified, installable APK.

All of the above was re-run from a clean clone of the DEPLOY bundle, not from the working copy that built it.